DEPI React training: discussion and revision guide

Saif eldin Ahmed Helmy · 420230168
Maaly Samir elsaied · 420230295
Individual discussions: Sunday, 1 November 2026, 09:00, according to the college email.

1. What to bring and how to use this pack

Bring your training certificate, the supplied course outline, your own presentation, and a copy of the team project. The email requires at least 10 slides per person and readiness to discuss both the course and projects. Each supplied deck contains 16 main slides and two optional backup slides. Each person presents independently. You do not split one presentation between you.

Open your PowerPoint in Presenter View to see the speaking notes. Read the notes to understand the material, then rehearse from the slide titles. Aim for roughly 10-12 minutes unless the examiner specifies another duration. Do not memorize every sentence. A natural answer has four parts: the concept, its purpose, an example from your work, and a limitation where relevant.

The explanations here describe the current repository inspected on 28 October 2026. It includes revisions after the original team PDF. Describe training-era work from your own recollection, and avoid implying that every later fix or test existed during the training. Individual responsibilities below come from your confirmation, rather than an inference from commit authorship.

2. Your project in one minute

The Interactive Learning Platform is a team-built web application prototype for learning frontend development. It organizes courses into modules and lessons. A learner can sign in, browse courses, open video lessons, attempt lesson quizzes, practice HTML/CSS/JavaScript in a browser preview, and view progress information. The profile and achievement screens make learning activity visible through indicators such as hours, streaks and earned badges.

The intended problem is passive video watching and fragmented practice across different tools. The platform puts learning and practice into one interface. This is a project rationale, not proof that the system improved educational outcomes.

Saif's opening: “I completed the DEPI React Frontend Web Developer training. In our team project, I was responsible for the backend and the main lesson page, including video playback and the organization of topics and subtopics. I will explain how the interface communicates with the server and

how lesson progress works.”

Maaly's opening: “I completed the DEPI React Frontend Web Developer training. In our team project, I worked on the landing page, profile and achievements. I will explain how these screens support the learner journey and how I used React state, API data and responsive layout.”

3. Architecture and a complete request

The client uses React 18, TypeScript, Vite, Bootstrap/React Bootstrap, React Router and Axios. The server uses Node.js, Express, TypeScript, Mongoose and MongoDB. Authentication uses express-session and a MongoDB-backed session store. It does not use JWT authentication in this checkout.

Consider opening a course:

1. React Router matches `/learn/:courseId` and renders `LessonPlayer` behind `ProtectedRoute`.
2. The page reads the course ID with `useParams` and calls the course service.
3. The shared Axios instance uses the configured API base URL and `withCredentials: true`.
4. Express matches `GET /api/courses/:id`. Session middleware has already read the session cookie.
5. Authentication middleware checks whether the user is logged in. The controller applies the relevant course access filter.
6. Mongoose queries the course and populates its module lesson references.
7. The server returns JSON containing a success flag and data.
8. React stores the response in state and renders the lesson list and selected content.

Frontend vs backend: the frontend handles browser interaction and presentation. The backend applies server-side rules and coordinates persistent data. A browser screen can guide users, but server checks must protect data because callers can make requests without using the interface.

Main data relationships

Entity	Purpose
User	Identity, role and stored learning indicators
Course	Course details and ordered modules
Module	A grouping within a course, with lesson references
Lesson	Course content, video URL, optional exercise and quiz reference
Quiz	Questions, options, correct indices, point weights and passing threshold
Progress	One user's progress in one course, including completed lessons and quiz attempts
Achievement / Badge	Achievement definitions and earned badge records

The Progress model has a unique compound index on userId and courseId. That expresses the intended rule of one progress document per learner per course. A schema describes stored data; an interface describes TypeScript expectations. Neither should be confused with a visual page template.

4. Saif's project revision

Lesson selection and modules

`LessonPlayer.tsx` keeps `currentModuleIndex` and `currentLessonIndex` in state. It uses those indices to get the selected lesson from `course.modules`. Clicking the sidebar updates the selection. Completed lesson IDs affect the completion markers. `showCodeEditor` switches between video and practice. These values change what the user sees, so state is appropriate.

The page fetches course details, attempts enrollment, requests the next lesson, and loads completed lessons. It uses services instead of putting every Axios call directly into the JSX. The UI checks previous lessons and quiz status before offering normal progression. This guidance should not be described as complete server-enforced anti-cheating protection.

Video lifecycle

`YouTubePlayer.tsx` receives `videoUrl`, `lessonId`, and `onVideoComplete` as props. It extracts the YouTube ID and creates a player through the YouTube IFrame API. When ready, it requests saved playback time and seeks there. It attempts to save the playback position every 10 seconds. At the end of the video, it invokes the parent callback. The parent decides whether to open a quiz or proceed.

Refs hold the player instance and interval handles. Updating a ref does not request a rerender. Effects initialize the external player and clean up timers and the player instance. This avoids leaving old resources active after navigation. Saved playback position means “where the video was,” not proof of focused learning.

Quiz and reward flow

The lesson quiz collects an array of selected option indices. The API route is `POST /api/quiz/:quizId/submit`. The controller loads the stored quiz, verifies its relationship with the supplied lesson and course, and calls `gradeQuiz`.

The grader validates answer count and option ranges. An answer of -1 represents an unanswered question. It sums question point weights and calculates the percentage. For example, earning 6 out of 10 weighted points gives 60%. The server compares the unrounded ratio with the passing threshold, so an apparently rounded percentage can differ from the exact pass comparison near a boundary.

The controller records attempts and uses a transaction to coordinate first-pass quiz rewards. A repeat pass should not repeatedly grant the same quiz points. Do not trust a score sent by the client; the server calculates the score from stored answers. The current quiz response still includes correct-answer information for client feedback, so server grading alone does not make this a secure examination system.

The separate `/quiz` page has local demonstration questions. It is different from `/lesson-quiz/:lessonId`, which loads a lesson-linked quiz from the API.

Sessions and roles

Registration hashes the password with bcrypt. Login checks the hash and stores userId and userRole in the session. A session cookie associates later requests with that server-side session. `HttpOnly` reduces direct access to the cookie from browser JavaScript. `sameSite: 'lax'` and secure cookies in production are part of the configuration. These settings do not establish that the whole application has undergone a security audit.

Authentication means identifying a user. Authorization means deciding what that user may do. Course creation uses authentication and teacher-role middleware. Administrative actions require the appropriate role. `ProtectedRoute` improves navigation but does not replace API authorization.

Files to revisit

Read `client/src/pages/LessonPlayer.tsx`, `client/src/components/YouTubePlayer.tsx`, `client/src/services/api.ts`, `server/src/server.ts`, `server/src/controllers/quizController.ts`, `server/src/controllers/progressController.ts`, and `server/src/config/session.ts`. Start with a named function and trace its caller and service; do not try to memorize every line in the repository.

5. Maaly's project revision

Landing page

`LandingPage.tsx` composes the hero, introductory sections and course summaries using Bootstrap layout and local project artwork. `useNavigate` connects the main call to action to the course route. Semantic sections and a clear headline help organize the content.

Its `COURSES` array is local placeholder content. That differs from the authenticated Courses page, which calls backend services. Do not describe the landing page course summaries as live database results. The existing marketing copy mentions an “intelligent guide,” but an implemented AI tutor is not present in the inspected code.

Profile loading and editing

`Profile.tsx` starts with loading state, calls `userService.getProfile()` in an effect, and stores the result as `profileData`. It renders account details and stored learning indicators. On a failed request, it offers a retry. An empty badges list has an explanatory message.

For name editing, `editingName` decides which UI appears and `newName` holds the draft. The input is controlled: its value comes from state, and onChange updates that state. Save rejects a blank trimmed value and calls `PUT /api/users/profile/name`. Success replaces the name in local profile state and exits edit mode. Cancel restores the saved name.

The update uses object spreading at both the profile and user levels. Copying only the outer object while mutating its nested user would still mutate the original nested object. The existing approach creates a new nested user object.

The profile displays treeLevel, but it does not itself calculate how a learner earns a new tree level. Login updates the streak in the backend. Stored study hours are not an independently verified record of focused study. Explain what the UI displays without inventing a progression formula.

Achievements

`Achievements.tsx` requests data through `achievementService`. It stores data, loading, error, and selectedCategory. The filtered list is derived from the selected category: all achievements or the matching group. Mapping the result produces cards with stable `_id` keys.

The backend combines achievement templates, existing badges and learning progress. Supported requirement types compare completed lesson/course counts or streaks with thresholds. It returns each item's earned status and current/required progress, grouped categories and summary totals. The current `all_courses` case remains incomplete.

Clicking a locked item shows how far the learner is from its requirement. Clicking an earned item shows an unlocked message. Clicking does not award the badge. The inspected page does not implement the old PDF's social-sharing feature or rarity tiers.

Files to revisit

Read `client/src/pages/LandingPage.tsx`, `Profile.tsx`, `Achievements.tsx`, `client/src/services/userService.ts`, `achievementService.ts`, and `server/src/controllers/achievementController.ts`. Trace initial loading, one successful interaction and one failure state in each dynamic screen.

6. Curriculum revision, topic by topic

The timetable provides subject names and hours, not a detailed lesson transcript. The following is a focused revision syllabus for those subjects. It does not claim your instructor covered every example below.

Prompt engineering - 6 hours

A useful prompt states a goal, relevant context, constraints and the required output. For coding, include the existing code or interface contract and ask for verification. Example: “Explain this React effect, identify its external dependency, and show what cleanup it needs.” Inspect generated code and test claims. Do not paste credentials or private user data into a prompt. Project link: this can support planning and review, but the repository does not prove exactly which AI tools the team used.

HTML5 essentials - 12 hours

HTML describes document structure. Use meaningful elements such as main, section, nav, heading, form, label, button and a. A button performs an action; an anchor represents navigation. Give inputs associated labels and images meaningful alt text when they communicate content. Form attributes can provide useful browser checks, but server validation remains necessary. Project link: the landing page sections and profile form.

CSS essentials - 9 hours

The box model is content, padding, border and margin. `box-sizing: border-box` includes padding and border within the declared dimensions. The cascade and selector specificity determine which declarations apply. Flexbox helps arrange items along a primary axis; Grid handles rows and columns together. Media queries adapt layout to available space. Project link: lesson layout, practice editor columns and Bootstrap styles. Being responsive requires checking actual screens; choosing Bootstrap alone does not prove every page works at every width.

Principles of UX/UI - 3 hours

UI concerns visible controls and presentation. UX concerns whether people can understand and complete tasks. Explain hierarchy, predictable navigation, readable labels, loading/error feedback and recovery. Accessibility includes keyboard operation, labels, visible focus and sufficient contrast. Project link: landing call to action, profile save/cancel, retry state and locked achievement feedback. Avoid claiming formal WCAG compliance without an audit.

JavaScript fundamentals - 15 hours

`const` prevents rebinding a variable; it does not freeze an object. `let` allows reassignment. Know primitive values, arrays, objects, functions, scope, closures and equality. `===` compares without the coercion performed by `==`. `map` transforms items, `filter` selects items and `reduce` combines values. Destructuring reads fields concisely; spread copies one level. Event handlers respond to interaction.

Promises describe eventual results. `async` functions return promises; `await` pauses that function's continuation while the rest of the application can continue. Use try/catch for request failure and finally for cleanup such as ending loading state. Project link: course fetching, profile saving and achievement filtering. A closure remembers variables from its surrounding scope; stale values in callbacks can explain unexpected state behaviour.

```
const names = courses.map(course => course.title);
const beginnerCourses = courses.filter(course => course.difficulty === 'beginner');
async function loadCourses() {
  try { return await courseService.getAllCourses(); }
  catch (error) { throw error; }
}
```

The catch above only rethrows for illustration. Real code should handle the failure where useful rather than add a redundant catch solely for appearance.

TypeScript - 6 hours

TypeScript statically checks JavaScript programs. Know type aliases, interfaces, unions, optional properties, generics and type narrowing. `User | null` means code must account for the absence of a user. `any` disables many useful checks. A type assertion tells the compiler what you believe; it does not validate data at runtime. Project link: Course/User interfaces and typed component props. Runtime validation uses tools such as Zod, express-validator or Mongoose validation at the appropriate boundaries.

```
interface LessonSummary { id: string; title: string; minutes: number }
function label(lesson: LessonSummary): string {
  return `${lesson.title} (${lesson.minutes} min)`;
}
```

Bootstrap - 12 hours

Bootstrap offers styles, utilities and a responsive grid. The grid uses containers, rows and columns, with 12 columns per row. A layout such as `col-12 col-md-6` occupies the full row on small screens and half from the medium breakpoint onward. React Bootstrap wraps Bootstrap patterns as React components. Project link: Row/Col/Card on the profile page, course cards and the navbar. Custom CSS still controls important project-specific layouts.

React - 33 hours

A component is a function that returns a description of UI, usually JSX. JSX resembles HTML but supports JavaScript expressions and React conventions such as className. Props are inputs passed by a parent. State is memory associated with a component's position in the rendered tree. Calling its setter requests rendering with an updated value.

Do not mutate state directly. Use a new array/object, and use a functional setter when the next state depends on the previous state. Rendering should stay pure. Event handlers perform interaction-driven work. Effects synchronize with external systems, such as a player, timer or API request. Declare dependencies based on values the effect uses and return cleanup for resources that need it.

`useRef` preserves a mutable reference without requesting a rerender. `useContext` reads a shared context value. A custom hook packages reusable hook logic. React Router chooses components according to the URL; useParams reads route parameters and useNavigate changes routes. Controlled forms use state for the input value. Keys help React identify list items across renders. A key is an identity hint, not a CSS selector or a value automatically passed as a prop.

```
function CategoryFilter({ categories }) {
  const [selected, setSelected] = useState('all');
  return <select value={selected} onChange={e => setSelected(e.target.value)}>
    <option value="all">All</option>
    {categories.map(category => <option key={category}>{category}</option>)}
  </select>;
}
```

Know why an effect without a dependency array runs after each commit, why dependencies can cause reruns, and why an empty dependency array is associated with mounting. Development Strict Mode may repeat effect setup and cleanup to expose mistakes. Do not fix that by removing necessary cleanup.

Introduction to backend: Node.js and Express - 9 hours

Node.js runs JavaScript outside the browser. Express maps HTTP methods and paths to handlers. Middleware can parse input, attach session information, reject unauthorized access or pass control with next(). A controller coordinates a request's work. A model represents database data and operations.

Know GET for reads, POST for creation/actions, PUT for updates under this API's conventions, and DELETE for removal. JSON is a data interchange format. Typical statuses: 200 successful request, 201 creation, 400 invalid input, 401 authentication required, 403 forbidden, 404 unavailable resource, 409 conflict, 500 server failure. CORS controls which browser origins may access responses; it is not a substitute for login or authorization. Project link: Express routes and Axios services.

Code style, patterns and best practices - 3 hours

Use names that explain intent, small responsibilities, consistent formatting and reusable components where they actually help. Keep request code in services and server responsibilities in routes/controllers/models. Handle errors at boundaries. Keep configuration separate from secrets. The smallest clear solution is preferable to unnecessary abstraction. Project link: shared Axios configuration and separate course/user services. The repository is a prototype and contains large components and incomplete cases; explain an improvement without claiming the architecture is perfect.

Containerization basics using Docker - 3 hours

An image is a packaged filesystem and metadata used to start a container. A container is an isolated running process. A Dockerfile describes an image build. Port mappings make a container's service reachable; volumes retain data outside a disposable container's writable layer. Environment variables configure processes. Compose can describe several services together. Project link: these concepts could package the frontend, API and database, but this repository does not contain a Dockerfile or Compose setup. Do not claim the application was containerized based only on the curriculum.

Functional documentation and unit testing - 3 hours

Documentation should explain setup, required configuration, expected inputs/outputs and limitations. A unit test isolates a small behaviour. An integration test checks collaboration between real parts, such as an API and database. An end-to-end test exercises the user journey. Arrange inputs, perform an action, then assert an observable result. A stub replaces an external dependency with controlled behaviour.

Project example: `gradeQuiz` can be tested with known questions and answer indices. A wrong answer should score zero. A database test can check transaction rollback and reward replay protection. In this preparation run, 11 backend tests passed and two database tests skipped. The frontend build succeeded, but a build is not a UI test and skipped tests are not passes.

Getting started with Git and GitHub - 3 hours

Git records snapshots called commits. The working tree is your current files, staging selects the next snapshot, and commit records it. A branch names a line of history. Pull retrieves/integrates remote changes and push publishes local commits. A merge combines histories and may require conflict resolution. A pull request lets teammates discuss a proposed change. `.gitignore` excludes

local/generated files from normal untracked-file staging; it does not remove an already tracked secret from history. Project link: this is a multi-contributor repository. Use “we” for shared work and “I” for your confirmed responsibility.

Capstone project - 3 scheduled contact hours

A capstone applies the curriculum to a larger problem. Be ready to explain the intended user, core journey, division of work, architecture, a concrete implementation and one limitation. The timetable's contact hours are not the team's total development time. Your own actual work and experience are the strongest evidence.

7. Likely examiner questions and model answers

Questions for either presenter

What did the training teach you? “It covered frontend foundations through React, with supporting topics such as TypeScript, Bootstrap, backend APIs, Git and testing. The project helped me connect those concepts to an application.” Follow with one example from your own screens.

Why React? “It lets us build the interface from reusable components and render it based on data and state. This project has shared navigation and dedicated pages with different local state.” Avoid claiming React is always the best choice.

Props versus state? “Props come from the parent. State is memory managed by the component. YouTubePlayer receives lessonId as a prop; the lesson page stores the selected lesson in state.”

What causes a rerender? “A state update, a context update, or rendering by a parent can lead to another render. Changing a ref by itself does not request one.”

What is useEffect for? “Synchronizing React with something external, such as loading profile data or managing a video player. Cleanup stops resources such as timers.”

Why TypeScript if we already validate forms? “TypeScript checks code before it runs. Form and server validation check actual runtime inputs. They solve different problems.”

Is this a finished production platform? “It is a training prototype. The main screens and API structure exist, but some analytics are placeholders, practice validation is basic, and fuller integration and accessibility checks remain.”

What is the difference between frontend and backend validation? “Frontend validation gives quick feedback. Backend validation protects the system when a caller bypasses the interface.”

How did you collaborate? State the actual division: Saif handled backend and lesson experience, Maaly handled landing/profile/achievements, and other team members contributed remaining parts. Only describe branching, meetings or review practices that you actually used.

What was your biggest challenge? Choose a real experience before rehearsal. Saif can consider synchronizing player state and saved progress if that was genuinely difficult. Maaly can consider keeping edit/cancel state consistent or handling API loading if that reflects her experience. Do not invent an incident, deadline, metric or fix.

Questions especially relevant to Saif

How does resume work? “The player loads saved currentTime for the lesson when ready, seeks there, and periodically saves the new position through the progress API.”

Why refs for the player and timers? “They preserve mutable external handles across renders without using them as visible UI state. Cleanup uses those handles to stop timers and destroy the old player.”

How do you prevent progress leaking between courses? “The API resolves the lesson's course and scopes the progress query to the current user and that course. The progress model also has a unique user/course index.”

Who decides the quiz score? “The server grades submitted answer indices against the stored quiz. The browser displays feedback, but its claimed score is not authoritative.”

Why a transaction for rewards? “Attempt storage and a first-pass points update should succeed together. The first-pass claim also prevents repeated passing attempts from granting the same quiz reward.” Mention that database tests were not run locally here.

Can somebody bypass a disabled Next button? “Yes, client controls can be bypassed. The backend needs to enforce every progression rule that matters. The current UI guidance is stronger than the server's full progression enforcement.”

What is the password storage method? “A bcrypt hash. Login compares the supplied password with the stored hash. We should not store plaintext passwords.”

Why sessions rather than JWTs? “This implementation uses a browser session cookie tied to server-stored session data. That matches the inspected code. I would explain the tradeoffs if redesigning authentication, but JWTs are not the mechanism here.”

Questions especially relevant to Maaly

What did you personally build? “The landing page, profile and achievements. I can show their layout, state and API integration. Backend and lesson playback were other team responsibilities.”

Are landing-page courses fetched from the database? “The landing page uses local course summaries. The authenticated catalog is a separate page that requests course data from the backend.”

How does the name field work? “newName controls the input. Save validates it and awaits the update request, then updates profileData. Cancel restores the saved name.”

Why copy nested objects? “React state should be updated without mutating the existing object. Copying the profile and its user creates a new value at both levels.”

How does achievement filtering work? “selectedCategory selects either all achievements or the corresponding grouped array. React derives and renders that list from the current state.”

Does clicking an achievement unlock it? “No. Clicking shows feedback. The backend computes earned status from supported progress requirements and existing badges.”

What if the API fails? “The profile and achievements render error feedback instead of normal data. The profile provides a Try Again action. Loading indicators explain the wait.”

Is the page fully accessible? “I used readable labels and feedback patterns, but I would not claim audited compliance. A complete keyboard and screen reader review is a further improvement.”

8. Differences from the original project PDF

Original deck wording	Accurate description for this checkout
Monaco code editor	Controlled textarea with iframe live preview
Real-time admin analytics/system status	Admin dashboard contains placeholder figures
Social badge sharing and rarity levels	Current page filters categories and shows earned/locked progress
Mandatory progression	The UI guides progression; complete server enforcement is not established
Tree/streak growth	Screens display stored indicators; login updates streaks and no complete growth claim is made
AI tutoring, mobile app and peer study groups	Future ideas, not implemented features

Code practice “tests” are expected-text checks against parsed HTML. They do not execute arbitrary JavaScript test cases. Some success messages mention rewards that the corresponding component does not itself persist. Explain these as prototype boundaries if asked.

9. Short demonstration and fallback

Saif, 90 seconds: course catalog (10 seconds), lesson sidebar and selection (25), video resume explanation (20), switch to code practice (20), explain quiz/progress request (15). A screenshot explanation is sufficient if the database or internet is unavailable. Do not spend the discussion troubleshooting setup.

Maaly, 90 seconds: landing call to action (20 seconds), profile fields and name edit flow (30), achievements filtering and locked feedback (30), explain the API boundary (10).

The screenshots in these decks show the actual local React application with intercepted sample API responses. They demonstrate the interface, not a functioning MongoDB session. The lesson screenshot includes an embedded YouTube HTML lesson. It is an illustrative sample course, not evidence of DEPI's own teaching videos.

For a real live run, follow the repository README: install the client/server dependencies, configure a local server environment privately, use a MongoDB replica set, start the server on its configured port and the Vite client on port 5173. Rehearse with an account and populated course data before the discussion. Keep credentials out of slides and screen sharing. The supplied screenshots avoid that setup dependency.

10. Rehearsal plan before Sunday

First pass, 45-60 minutes: understand sections 2-5, trace your own pages, then deliver your main slides aloud once. Mark concepts you cannot explain without notes.

Second pass, 60 minutes: revise HTML/CSS/JavaScript/TypeScript/React, then explain one project example for each. Review the shorter curriculum subjects so none is unfamiliar.

Third pass, 30 minutes: ask each other the questions in section 7. Answer in your own words. If you do not know something, explain what you do know and how you would verify the rest.

Final pass, 20 minutes: time a complete solo presentation, practice one interruption, and rehearse the screenshot fallback. Copy the PowerPoint, PDF and course outline to a USB drive and keep another copy available. Bring the training certificate and project as requested by the college.

A useful transition is: “That is the user-facing behaviour. I will now explain the state and API request behind it.” Another is: “This part belongs to the shared project; my own contribution was …”

11. Sources and code navigation

Evidence: the college email supplied in chat, the course outline image, the 15-page team project PDF, the user's confirmed responsibilities, and the current repository. Source references are also in each slide's speaker notes.

Primary documentation used to verify revision concepts:

- React state: https://react.dev/learn/state-a-components-memory
- React effects: https://react.dev/learn/synchronizing-with-effects
- TypeScript static checking: https://www.typescriptlang.org/docs/handbook/typescript-from-scratch
- Bootstrap 5.3 grid: https://getbootstrap.com/docs/5.3/layout/grid/
- Express middleware: https://expressjs.com/en/guide/using-middleware/
- Docker container concepts: https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/
- JavaScript revision: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting

Treat these as revision references rather than a record of exactly what the instructor said. The supplied curriculum image is the evidence for the course subjects and hours.